

# Customer Churn Prediction & LTV Engine

Week 4 • Day 4

## Docker Containerisation

Package the entire application in Docker containers for portable deployment

### ■ Day Objectives

- ◆ Understand Docker: images, containers, Dockerfiles, and docker-compose
- ◆ Create a Dockerfile for the FastAPI application
- ◆ Create a docker-compose.yml for the full stack (API + PostgreSQL)
- ◆ Build and run the containerised application
- ◆ Verify the containerised API works identically to the local version

## ■ 1. Docker Explained from Scratch

**Docker** is a platform that packages applications and all their dependencies into a standardised unit called a **container**. A container runs the same way on any computer — Windows, Mac, Linux, cloud server. It eliminates 'works on my machine' problems.

### Key Docker Concepts

- **Image:** A read-only blueprint for a container (like a class in OOP). Built from a Dockerfile.
- **Container:** A running instance of an image (like an object). Isolated from the host.
- **Dockerfile:** A text file with instructions to build an image.
- **docker-compose:** A tool to define and run multi-container applications (API + DB + dashboard).

- **Volume:** A way to persist data beyond the container's lifetime (our PostgreSQL data).

## ■ 2. Docker Setup

### Install Docker

Go to [docker.com/get-docker](https://docker.com/get-docker) and download Docker Desktop for your OS. Install and start it. Verify with: `docker --version`

#### ■ Dockerfile – API container

```
# ■■ Base image: Python 3.11 slim
FROM python:3.11-slim

# ■■ Set working directory inside container
WORKDIR /app

# ■■ Copy requirements and install (cached layer)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ■■ Copy all source code
COPY . .

# ■■ Expose port 8000
EXPOSE 8000

# ■■ Start command
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

#### ■ docker-compose.yml – full stack

```
version: '3.8'

services:
  # ■■ PostgreSQL database
  postgres:
    image: postgres:15
    container_name: churn_postgres
    environment:
      POSTGRES_DB: churn_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres123
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - '5432:5432'
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U postgres']
      interval: 10s
      timeout: 5s
      retries: 5

  # ■■ FastAPI ML service
  api:
```

```

build: .
container_name: churn_api
environment:
  DB_HOST: postgres
  DB_PORT: 5432
  DB_NAME: churn_db
  DB_USER: postgres
  DB_PASSWORD: postgres123
volumes:
  - ./models:/app/models
  - ./data:/app/data
ports:
  - '8000:8000'
depends_on:
  postgres:
    condition: service_healthy

volumes:
  postgres_data:

```

#### ■ Build and run with Docker Compose

```

# Build the images
docker-compose build

# Start all services
docker-compose up -d

# Check containers are running
docker-compose ps

# Test the API
curl http://localhost:8000/health

# View logs
docker-compose logs api

# Stop everything
docker-compose down

```

## ■ 3. Commit & Checklist

- Dockerfile created
- docker-compose.yml created
- docker-compose up starts both containers
- curl http://localhost:8000/health returns healthy

#### ■ Week4-Day4 Commit

```

git add .
git commit -m "Week4-Day4: Docker containerisation – Dockerfile + docker-compose for full stack"
git push origin main

```

■ Estimated Time: 5 – 7 hours

— End of Week 4 Day 4 — Zaalima Development Confidential —